CMPE 258 HW1 Report — Language Models from Scratch (Part 1)

Aaron Chandler · CMPE 258, Fall 2026 · November 8, 2026

1. Question and hypothesis

This report measures what the standard language-model metrics mean and how recurrent networks and Transformers differ in practice on a small character-level task. Hypotheses, stated before running:

1. **Metrics.** Perplexity, bits/token and bits-per-byte (bpb) are deterministic transforms of the mean cross-entropy loss, so recomputing them from `val_loss`, `val_tokens` and `val_bytes` should match the logged values exactly. An untrained model should score about ln V, the loss of a uniform guess over V tokens.
2. **Architectures.** The vanilla RNN should trail far behind, because gradients through its repeated tanh updates vanish and it cannot learn longer-range structure. Gated RNNs (GRU, LSTM) should close most of that gap. A modern Transformer (RMSNorm + RoPE + SwiGLU) should be best at an equal token budget.
3. **Model size.** A 4× larger LSTM may overfit, since the corpus is only 1 MB and is seen about 16 times.
4. **Context.** The Transformer should benefit from longer context more than the LSTM, because attention reads any earlier position directly while the LSTM must carry everything in one fixed-size state.
5. **Attention.** Individual heads should specialize in simple, measurable patterns (previous character, start of the current word, line/speaker markers).

2. Setup

Every experiment is a config or sweep file run with the `nanolm` library; all of them are in the submission's `configs/` folder. No library code was edited.

Item	Value
Data	Tiny Shakespeare, 1,115,394 characters; the last 10% (111,540 characters) is the fixed validation split
Tokenizer	Characters, V = 65 (Task 1 also uses bytes, V = 256, and GPT-2 BPE, V = 50,257)
Token budget	2,000 steps × 8,192 tokens/step = 16.4M tokens (about 16 passes over the training set), identical for every model
Context	Tasks 2 and 4: 256 (the Task 1 L1 LSTM uses 128). Task 3: 64, 128, 256, 512 with batch size 128, 64, 32, 16, so tokens per step stay at 8,192
Evaluation	The full validation split, each token scored once; metric = validation bpb
Seeds	1337 and 42 (Task 3 also 7); the same seeds for every model, so differences are paired
Device / code	NVIDIA GeForce RTX 5070 Ti, bf16 autocast, PyTorch 2.11.0+cu128, conda env (Python 3.11); `nanolm` from DeepDataMiningLearning commit `8418fc1bae85657190f11749483bf2e729b9702c` (its only uncommitted changes were my two added sweep files)

Training compute uses FLOPs ≈ 6 × N × D (N = parameters, D = 16.4M training tokens); for the RNNs this is a rough guide only. Each run trained in about 4–25 seconds; the whole homework took under 10 minutes of training time (summed `train_time_sec` from `metrics.json`).

Model	Layers × width	Parameters	Training FLOPs
Vanilla RNN	2 × 256	0.28M	2.8×10^{13}
GRU	2 × 256	0.81M	7.9×10^{13}
LSTM	2 × 256	1.07M	1.1×10^{14}
LSTM (large)	2 × 512	4.24M	4.2×10^{14}
Transformer, GPT-2 style (LayerNorm, learned positions, GELU)	4 × 256, 4 heads	3.44M	3.4×10^{14}
Transformer, modern (RMSNorm, RoPE, SwiGLU, no bias)	4 × 256, 4 heads	3.16M	3.1×10^{14}

3. Results

3.1 Metrics by hand (Task 1)

All four metrics follow from three logged numbers: the mean loss $\mathcal{L}$ (nats/token), the number of scored tokens and the number of bytes they cover:

$$\text{PPL} = e^{\mathcal{L}} \quad \cdot \quad \text{bits/token} = \mathcal{L} / \ln 2 \quad \cdot \quad \text{bpb} = (\mathcal{L} / \ln 2) \times (N_{\text{tokens}} / N_{\text{bytes}})$$

Trained LSTM 2×256 (seed 1337)	By hand	Logged
val_loss (input)	1.492449	—
val_tokens / val_bytes (input)	111,488 / 111,488	—
Perplexity = $e^{1.492449}$	4.447977	4.447977
Bits/token = 1.492449 / 0.693147	2.153149	—
bpb = 2.153149 × (111,488 / 111,488)	2.153149	2.153149

The hand values match the logs to within 1e-15. A perplexity of 4.45 means the model is, on average, as uncertain as a fair choice among about 4.5 of the 65 characters. For characters on ASCII text one token is one byte, so bpb equals bits/token.

Untrained models. `*_init` runs use learning rate 0, so the weights stay at their random initialization; `*_step1` runs take one optimizer step (the assignment's `train.max_steps=1`). Same modern Transformer, seed 1337:

Tokenizer	V	ln V (nats)	Loss, init	Loss, 1 step	Init − ln V	Bytes/token	bpb, init
char	65	4.1744	4.2166	4.1602	+0.042	1.00	6.08
byte	256	5.5452	5.5118	5.4434	−0.033	1.00	7.95
GPT-2 BPE	50,257	10.8249	10.8216	10.7926	−0.003	3.09	5.05

Every initialized model scores within 1% of ln V, and a single optimizer step already moves each one below it.

3.2 RNN vs. GRU vs. LSTM vs. Transformer (Task 2)

The modern Transformer is best (2.109 bpb) and the vanilla RNN is worst by a wide margin (2.444). Same data, 16.4M tokens each, `configs/sweeps/01_char_models.yaml`, 2 seeds; Δ and p are paired against the GPT-2-style Transformer.

Model	Params	Val bpb (mean ± std)	Δ vs. GPT-2 style	Paired p	Final train / val loss (nats)	Best val loss
Transformer, modern	3.16M	**2.1088 ± 0.0071**	−0.0716	0.021	1.199 / 1.462	1.462
LSTM 2×256	1.07M	2.1356 ± 0.0017	−0.0447	0.021	1.289 / 1.480	1.480
GRU 2×256	0.81M	2.1650 ± 0.0010	−0.0153	0.080	1.345 / 1.501	1.501
LSTM 2×512	4.24M	2.1709 ± 0.0082	−0.0095	0.464	1.024 / 1.505	1.479
Transformer, GPT-2 style	3.44M	2.1803 ± 0.0038	0	—	1.355 / 1.511	1.511
Vanilla RNN 2×256	0.28M	2.4442 ± 0.0008	+0.2639	0.008	1.629 / 1.694	1.694

Seed noise floor (median within-model std): 0.0027 bpb, so differences below about 0.005 bpb are treated as not resolved. With only 2 seeds, a paired p-value rests on one degree of freedom and is a rough guide. All six results are within 0.01 bpb of the reference runs published in the `nanolm` tutorial (e.g. 2.105, 2.134 and 2.436 for the modern Transformer, LSTM and RNN), which is evidence that the setup is correct.

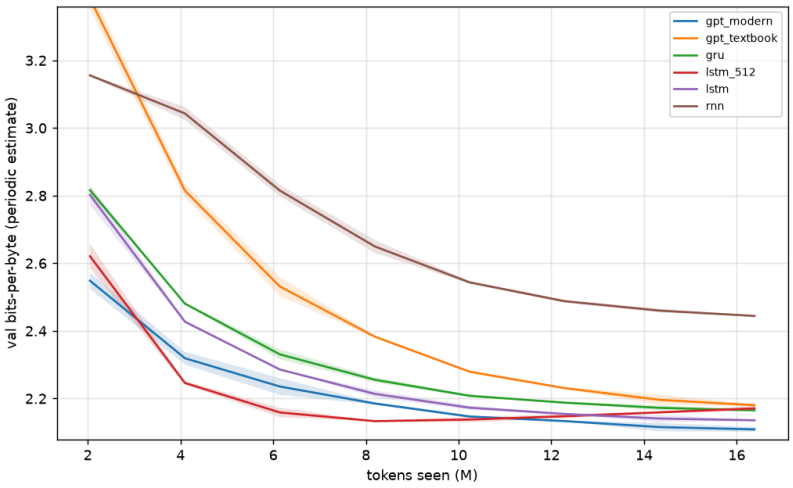


Figure 1. Validation bpb vs. training tokens (millions), mean over 2 seeds, shaded ± std. The vanilla RNN flattens high; LSTM 2×512 (red) bottoms out near 8M tokens and then rises.

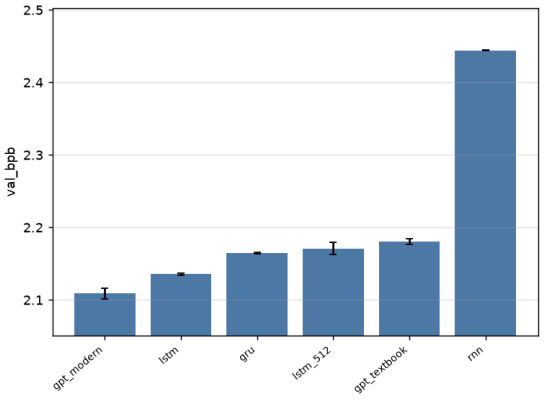


Figure 2. Final validation bpb per model, mean ± std over 2 seeds (lower is better; the y-axis starts at 2.05).

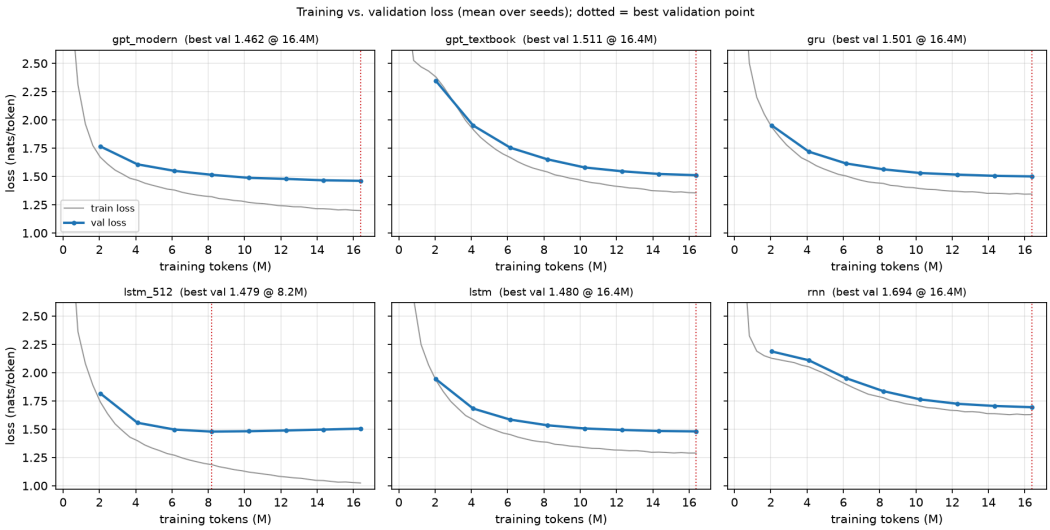


Figure 3. Training loss (grey) vs. validation loss (blue) in nats/token; the dotted line marks the best validation point. Only LSTM 2×512 peaks early (8.2M tokens) while its training loss keeps falling.

3.3 Context length (Task 3)

The Transformer is better at every context length. Its gain stops being measurable after 128 characters; the LSTM's stops after 256. `configs/sweeps/hw1_context.yaml`, 3 seeds (1337, 42, 7), fixed budget of 8,192 tokens per step (16.4M total):

seq_len	Transformer bpb	LSTM bpb	Transf. gain	LSTM gain
64	2.1605 ± 0.0035	2.1834 ± 0.0047	—	—
128	2.1204 ± 0.0070	2.1523 ± 0.0047	+0.040 (res.)	+0.031 (res.)
256	2.1128 ± 0.0015	2.1375 ± 0.0029	+0.008 (n.r.)	+0.015 (res.)
512	2.0980 ± 0.0113	2.1287 ± 0.0055	+0.015 (n.r.)	+0.009 (n.r.)
Total 64→512			+0.062	+0.055

Gain = improvement over the previous length. A gain is resolved (res.) when it exceeds 2× the larger seed std of the two lengths compared; otherwise not resolved (n.r.). The Transformer at 512 is the noisiest cell (std 0.011). Adding a third seed changed one verdict: the LSTM's 256→512 step was resolved with 2 seeds and is not with 3. Training time per run was 14 s for the Transformer at every length but grew from 7 s to 24 s for the LSTM, whose time loop is sequential.

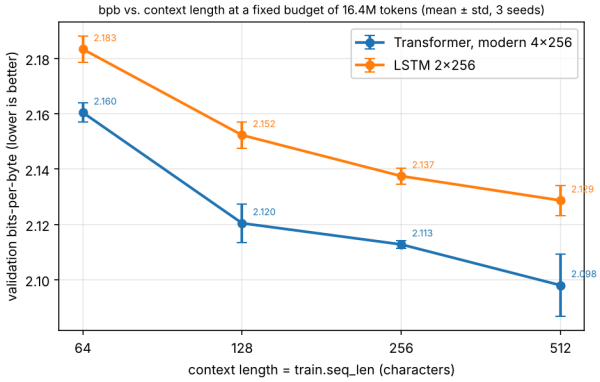


Figure 4. Validation bpb vs. training context length (log-2 axis), mean ± std over 3 seeds, fixed budget of 16.4M tokens.

3.4 Attention heads (Task 4)

Three head types stand out, each at 5–8× the weight a uniform head would give. Model: the modern Transformer from Task 2 (seed 1337, 4 layers × 4 heads). Maps from `python -m nanolm.viz attention`; numbers from my script `scripts/head_scores.py`.

Method. For each head and query position t, I average the attention weight on one target position: t−1 (previous character), the first letter of t's word (only for letters at least 2 into a word, so it never overlaps with t−1), the latest `:` or newline before t−1, and position 0. Chance is what a head spreading evenly over positions 0..t would give, 1/(t+1), averaged over the same positions. Texts: dialogue `ROMEO:⏎But soft! what light through yonder window` (in-domain); prose `The engineer checked the robot arm and the motor.` (modern words); repeat `the king, the king, the king, the queen.` (structured).

Head	Role	Dialogue	Prose	Repeat
L2·H2	Previous character	0.609 (8.4×), look-back 3.2	0.585 (8.1×), look-back 2.5	0.546 (6.5×), look-back 2.7
L3·H3	First letter of word	0.357 (5.5×)	0.158 (2.2×)	0.593 (7.6×)
L3·H2	Latest `:` / newline	0.376 (8.4×); position 0: 0.046	no marker; position 0: **0.265 (4.2×)**	no marker; position 0: 0.040 (0.5×)

Weight on the head's target, × chance; look-back = mean distance attended, in characters. L2·H2 behaves the same on every text. L3·H3's word-start pattern is strongest on the repeated words and weakest on modern prose. L3·H2 has the longest look-back of all 16 heads on dialogue and

prose (12.5 and 12.9 characters). The strongest head of each type on each text is listed in the appendix.

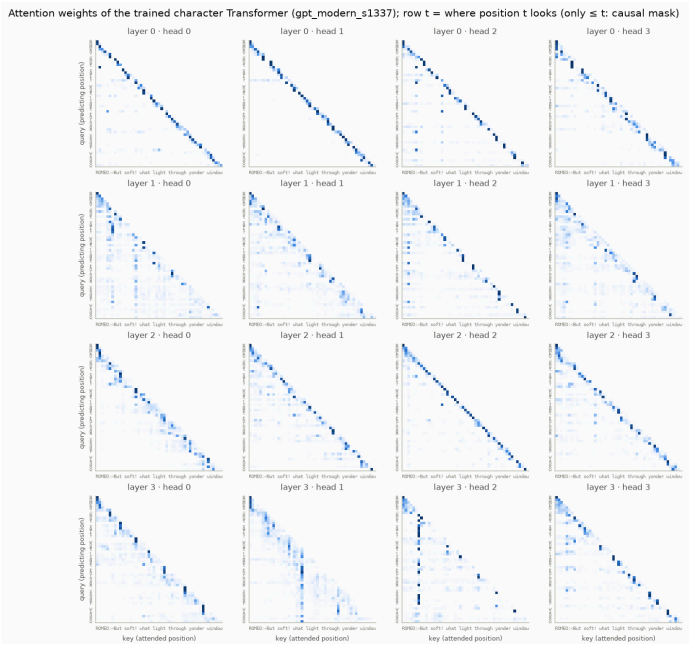


Figure 5. All 16 attention maps on the dialogue text; row t = where position t looks. L2·H2 is a sharp line just below the diagonal (previous character); L3·H2 is a vertical stripe at the `:`/newline after `ROMEO`.

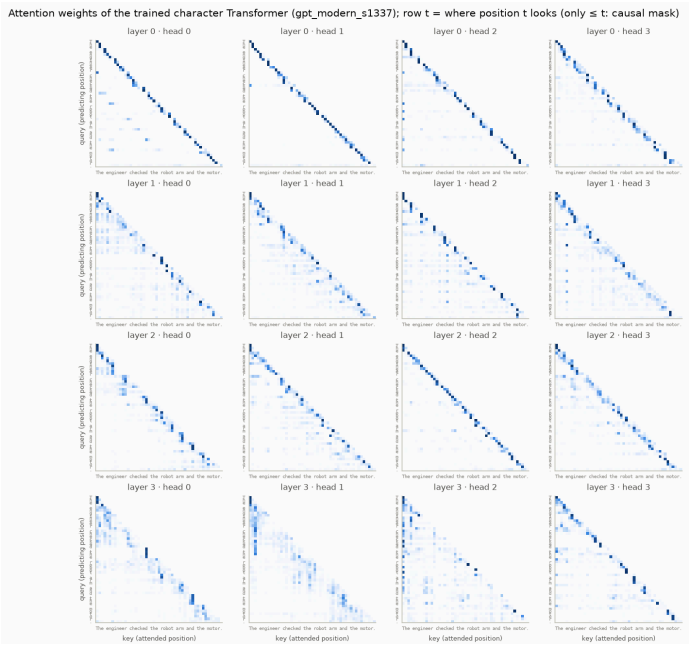


Figure 6. The same model on the prose text, which has no `:` or newline: L3·H2 now puts its weight on position 0 (the first column).

4. Analysis

Metrics. The four metrics are one measurement in different units, confirmed exactly in §3.1. The untrained models confirm that a random initialization is close to uniform: loss ≈ ln V. The char model sits slightly *above* ln V (+0.042): random weights are not perfectly uniform, so the model is slightly confidently wrong. The byte model sits *below* ln 256 even at initialization (−0.033). The 65 characters of the corpus are only 65 of the 256 byte values, and with tied input/output embeddings an untrained model leans toward repeating the current token, which is always one of those 65. That moves a little probability off the 191 bytes that never occur. One optimizer step lowers every loss further, plausibly because the first thing learned is which tokens are common. The untrained GPT-2-tokenizer model shows why bpb is needed: its loss is 2.6× the char model's, yet its bpb is lower (5.05 vs. 6.08), because each of its tokens covers 3.09 bytes. Per-token loss and perplexity cannot compare tokenizers; bpb can.

Q1. Why does the vanilla RNN fall so far behind? It *underfits*: it fails to learn the patterns, rather than memorizing. Its training loss stays high (1.629 nats) and its train/val gap is the smallest of all models (0.065), so the problem is what it can represent, not overfitting. The mechanism is the vanishing gradient. The state is overwritten through a tanh every step, so the error signal reaching k steps back is multiplied by k Jacobians with norm below 1 and shrinks exponentially, and the RNN learns little beyond short-range character statistics. The evidence is a controlled comparison: the GRU and LSTM differ from it mainly by gates (an additive, gated memory path), with the same data, budget and width, and they close 0.28–0.31 bpb of the gap. Some of that gap may also come from the RNN's smaller size (0.28M parameters), but the GRU and LSTM recover far more than their extra parameters would explain.

Q2. Is the larger LSTM better or worse? Worse at the end of training: 2.1709 vs. 2.1356 bpb for the 2×256 LSTM, despite 4× the parameters. The curves show overfitting. Its training loss falls to 1.024 nats while validation ends at 1.505, a train/val gap of 0.480, the largest of any model (Figure 3). Its validation loss reached its best value (1.479) at 8.2M tokens and then rose. At that best point it only ties the small LSTM's final 1.480, so the extra capacity bought nothing on 1 MB of text seen about 16 times. Early stopping or more dropout would recover the best checkpoint; more data is the real fix.

Q3. Which architecture uses longer context more effectively? The Transformer, but mostly by using *short* context better, and the hypothesis is only partly supported. It is 0.023 bpb ahead already at 64 characters, gains a resolved 0.040 from 64 to 128, and stays 0.025–0.032 ahead at every length (2.098 vs. 2.129 at 512). Its later gains (+0.008, +0.015) are not resolved, so it stops measurably benefiting after 128 characters. The LSTM keeps benefiting one doubling longer (+0.031, +0.015, both resolved) and stops after 256. Over 64→512 the two gain about the same in total (0.062 vs. 0.055). So the Transformer extracts more from the first 128 characters, and the LSTM's later gains are partly catching up. I treat this as a genuine negative result for the hypothesis, not a bug: (1) the 256-length results reproduce Task 2 within noise; (2) the data may not reward long context, since most of what

predicts the next character in Shakespeare dialogue sits within the current line, usually under 100 characters; (3) the evaluation limits the benefit, since windows are cut end to end and a token at window length T sees on average only T/2 characters of context. The Transformer's 512 result is also noisy (std 0.011); sliding-window evaluation, where every scored token has long context, would test the claim more directly.

Attention heads. Heads specialize in measurable roles (§3.4). A previous-character head (L2·H2, 0.61 weight, 8.4× chance) acts as a learned bigram detector. A word-start head (L3·H3, 0.36, 5.5×; 0.59 on the repeated text) lets the model spell out the rest of a word from its beginning. A marker head (L3·H2, 0.38, 8.4×) tracks where the current line of dialogue began. The prose text exposes a further behavior: it contains no `：` or newline, and L3·H2 then puts 0.265 of its weight on position 0 (4.2× chance). When its target is missing, the head defaults to the first token, behavior known as an attention sink. These roles and head numbers match the tutorial's reference model (L3·H2 marker head, L3·H3 word-start head), probably because both used seed 1337.

Bug or genuine result? No result contradicts its hypothesis in a way that suggests a bug: all six Task 2 numbers are within 0.01 bpb of the tutorial's reference runs, hand-computed metrics match the logs exactly, and the untrained losses sit at ln V. A useful reproducibility check is that Task 2's modern Transformer and Task 3's `gpt_modern_T256` are the same configuration and seeds. On the same two seeds they gave 2.1088 and 2.1122 bpb (and the LSTM pair 2.1356 vs. 2.1359): run-to-run GPU nondeterminism, within the seed noise.

5. Conclusion and limitations

At an equal budget of 16.4M characters, the modern Transformer is the best character model (2.109 bpb), gated RNNs follow (LSTM 2.136, GRU 2.165), and the vanilla RNN trails by 0.26–0.34 bpb because it cannot learn beyond short-range patterns. The training recipe matters as much as the architecture: the GPT-2-style Transformer is worse than a small LSTM. Bigger is not automatically better: the 4× larger LSTM overfits the 1 MB corpus. The metrics behave exactly as defined, and only bpb compares models with different tokenizers.

Limitations: one small corpus (1 MB of Shakespeare), so conclusions about overfitting and context may not hold with more data; two seeds for Task 2 (three for Task 3), so paired p-values rest on very few degrees of freedom; one size per architecture, each with its tutorial learning rate rather than a tuned one; evaluation windows equal the training length, so a longer `seq_len` changes both training and the context available at test time; the attention analysis uses one model (one seed) and three short texts.

Appendix A: reproducibility

Commands to reproduce every result, run from the root of the submission folder with the `nanolm` library (commit above) importable:

```
python -m nanolm.train -c configs/laptop/L1_char_lstm.yaml                      # Task 1, trained model
python -m nanolm.sweep --sweep configs/sweeps/hw1_untrained.yaml                # Task 1, untrained models (6
runs)
python -m nanolm.sweep --sweep configs/sweeps/01_char_models.yaml               # Task 2 (12 runs)
python -m nanolm.sweep --sweep configs/sweeps/hw1_context.yaml --seeds 1337 42 7 # Task 3 (24 runs)
python -m nanolm.analyze <sweep output folder> --metric val_bpb               # summary.csv / summary.md
python -m nanolm.viz attention --run <Task 2 gpt_modern, seed 1337> --text "..."   # Task 4 maps
python scripts/head_scores.py --run <same run> --text "..." --out <file>.csv      # Task 4 numbers
```

In the submission folder	Contents
`configs/`	Every config used (same layout as `nanolm/configs`); my own are `sweeps/hw1_untrained.yaml` and `sweeps/hw1_context.yaml`
`results/<experiment>/summary.csv`, `summary.md`	Per-run metrics and seed statistics from `nanolm.analyze`
`results/<experiment>/runs/<run>/`	Each run's `metrics.json`, `meta.json` and `config.yaml` (no checkpoints)
`figures/`	All figures, including the attention maps
`scripts/`	Analysis and plotting scripts used for this report
`environment.yml`, `environment.lock.yml`	Conda environment recipe and exact versions

Appendix B: strongest head of each type per text

Head type	Dialogue	Prose	Repeat
Previous character	L2·H2: 0.609 (8.4×)	L0·H1: 0.609 (8.4×)	L2·H2: 0.546 (6.5×)
First letter of word	L3·H3: 0.357 (5.5×)	L1·H1: 0.263 (3.7×)	L2·H3: 0.624 (8.0×)
Latest `:` / newline	L3·H2: 0.376 (8.4×)	no marker in text	no marker in text
Position 0	L1·H3: 0.072 (1.1×)	L3·H2: 0.265 (4.2×)	L3·H1: 0.057 (0.8×)

Weight on the target position, with the ratio to the uniform-chance weight in parentheses.